

LABORATORY EXPERIMENT REPORT

Object Oriented Programming Laboratory

Experiment Number	01 / String Methods
Date	_____
Title	Demonstration of Core Java String Methods
Subject	Object Oriented Programming / Java Programming Laboratory
Student Name	_____
Roll No. / Class	_____

1. Aim / Objective

To study, implement and demonstrate the fourteen core methods of the **java.lang.String** class in Java. The experiment covers character access, case conversion, comparison, concatenation, substring extraction, searching, replacement, whitespace handling and prefix/suffix checking. Each method is illustrated with a working program, sample output and verified test cases.

2. Software & Hardware Requirements

Hardware

- Any standard desktop / laptop computer (Intel/AMD processor, minimum 4 GB RAM)

Software

- Java Development Kit (JDK) 8 or higher (OpenJDK / Oracle JDK)
- Any Java IDE or text editor (VS Code, Eclipse, IntelliJ IDEA, NetBeans, or Notepad++)
- Command-line tools: `javac` (compiler) and `java` (runtime)
- Operating System: Windows / Linux / macOS

3. Theory / Concept

In Java, the **String** class (package `java.lang`) represents an immutable sequence of characters. Once a String object is created, its content cannot be changed; any method that appears to modify a String actually returns a new String object. String objects are stored in the String constant pool, which enables efficient memory usage for identical literals.

The methods demonstrated in this laboratory belong to the following functional groups:

- **Character Access:** `charAt()`
- **Case Conversion:** `toUpperCase()`, `toLowerCase()`
- **Comparison:** `equals()`, `equalsIgnoreCase()`
- **Concatenation:** `concat()`
- **Extraction:** `substring()`
- **Searching:** `indexOf()`, `lastIndexOf()`, `contains()`
- **Modification (returns new String):** `replace()`, `trim()`
- **Prefix / Suffix Testing:** `startsWith()`, `endsWith()`

4. Algorithm (General Procedure)

1. Create a Java source file containing a public class with a main method.
2. Declare one or more String variables and initialize them with literal values.
3. Invoke the target String method with the appropriate arguments.
4. Store the returned value (char / String / boolean / int) in a suitable variable.
5. Display the original String(s) and the result using System.out.println().
6. Compile the program with `javac ClassName.java`.
7. Execute with `java ClassName` and verify the output against expected results.
8. Repeat the above steps for each of the 14 methods, writing a separate demonstration class.

5. UML Class Diagram (Conceptual)

The following conceptual diagram shows the String class and the methods exercised in this experiment. (Simplified; actual java.lang.String contains many more members.)

```
+-----+
|           java.lang.String           |
+-----+
| - value : char[]                     |
| - hash : int                         |
+-----+
| + charAt(index : int) : char          |
| + toUpperCase() : String              |
| + toLowerCase() : String              |
| + equals(anObject : Object) : boolean |
| + equalsIgnoreCase(anotherString : String) |
| + concat(str : String) : String       |
| + substring(beginIndex : int) : String |
| + indexOf(ch : int) : int             |
| + lastIndexOf(ch : int) : int         |
| + replace(oldChar : char, newChar : char) |
| + trim() : String                    |
| + contains(s : CharSequence) : boolean |
| + startsWith(prefix : String) : boolean |
| + endsWith(suffix : String) : boolean  |
+-----+
```

6. Source Code / Program

Fourteen independent demonstration programs are provided below. Each program focuses on a single method.

1. charAt() Method

Aim: To find the character present at a specified index in a String.

Definition: The charAt() method returns the character present at the specified index of a String. Indexing starts from 0.

Syntax: string.charAt(index);

Program:

```
class CharAtExample {  
    public static void main(String[] args) {  
        String str = "Hello Java";  
        char ch = str.charAt(6);  
        System.out.println("String: " + str);  
        System.out.println("Character at index 6: " + ch);  
    }  
}
```

Sample Output:

```
String: Hello Java  
Character at index 6: J
```

Test Case Verification:

Test Case	Input String	Index	Expected	Status
1	Hello	1	e	Pass
2	Java	0	J	Pass
3	Computer	3	p	Pass

Conclusion: The Java program was successfully executed and the charAt() method was demonstrated correctly with all test cases passing.

2. toUpperCase() Method

Aim: To convert a String into uppercase letters.

Definition: The toUpperCase() method converts all lowercase characters in a String into uppercase characters and returns a new String.

Syntax: string.toUpperCase();

Program:

```
class UpperCaseExample {  
    public static void main(String[] args) {  
        String str = "hello java";  
        String result = str.toUpperCase();  
        System.out.println("Original String: " + str);  
        System.out.println("Uppercase String: " + result);  
    }  
}
```

Sample Output:

```
Original String: hello java  
Uppercase String: HELLO JAVA
```

Test Case Verification:

Test Case	Input String	Expected Output	Actual Output	Status
1	hello	HELLO	HELLO	Pass
2	java programming	JAVA PROGRAMMING	JAVA PROGRAMMING	Pass
3	computer	COMPUTER	COMPUTER	Pass

Conclusion: The Java program was successfully executed and the toUpperCase() method was demonstrated correctly with all test cases passing.

3. toLowerCase() Method

Aim: To convert a String into lowercase letters.

Definition: The toLowerCase() method converts uppercase characters in a String into lowercase characters and returns a new String.

Syntax: string.toLowerCase();

Program:

```
class LowerCaseExample {  
    public static void main(String[] args) {  
        String str = "HELLO JAVA";  
        String result = str.toLowerCase();  
        System.out.println("Original String: " + str);  
        System.out.println("Lowercase String: " + result);  
    }  
}
```

Sample Output:

```
Original String: HELLO JAVA  
Lowercase String: hello java
```

Test Case Verification:

Test Case	Input String	Expected Output	Actual Output	Status
1	HELLO	hello	hello	Pass
2	JAVA PROGRAMMING	java programming	java programming	Pass
3	COMPUTER	computer	computer	Pass

Conclusion: The Java program was successfully executed and the toLowerCase() method was demonstrated correctly with all test cases passing.

4. equals() Method

Aim: To compare two Strings based on their contents.

Definition: The equals() method compares two Strings based on their contents. It returns true if both Strings contain the same characters; otherwise false.

Syntax: string1.equals(string2);

Program:

```
class EqualsExample {  
    public static void main(String[] args) {  
        String str1 = "Java";  
        String str2 = "Java";  
        boolean result = str1.equals(str2);  
        System.out.println("String 1: " + str1);  
        System.out.println("String 2: " + str2);  
        System.out.println("Strings are equal: " + result);  
    }  
}
```

Sample Output:

```
String 1: Java  
String 2: Java  
Strings are equal: true
```

Test Case Verification:

Test Case	String 1	String 2	Expected	Status
1	Java	Java	true	Pass
2	Hello	World	false	Pass
3	ABC	ABC	true	Pass

Conclusion: The Java program was successfully executed and the equals() method was demonstrated correctly with all test cases passing.

5. equalsIgnoreCase() Method

Aim: To compare two Strings without considering case differences.

Definition: The equalsIgnoreCase() method compares two Strings while ignoring differences between uppercase and lowercase letters.

Syntax: string1.equalsIgnoreCase(string2);

Program:

```
class EqualsIgnoreCaseExample {  
    public static void main(String[] args) {  
        String str1 = "Java";  
        String str2 = "JAVA";  
        boolean result = str1.equalsIgnoreCase(str2);  
        System.out.println("String 1: " + str1);  
        System.out.println("String 2: " + str2);  
        System.out.println("Strings are equal: " + result);  
    }  
}
```

Sample Output:

```
String 1: Java  
String 2: JAVA  
Strings are equal: true
```

Test Case Verification:

Test Case	String 1	String 2	Expected	Status
1	Java	JAVA	true	Pass
2	Hello	hello	true	Pass
3	Java	Python	false	Pass

Conclusion: The Java program was successfully executed and the equalsIgnoreCase() method was demonstrated correctly with all test cases passing.

6. concat() Method

Aim: To join two Strings.

Definition: The concat() method joins the specified String to the end of another String and returns the combined result.

Syntax: string1.concat(string2);

Program:

```
class ConcatExample {  
    public static void main(String[] args) {  
        String str1 = "Hello ";  
        String str2 = "Java";  
        String result = str1.concat(str2);  
        System.out.println("First String: " + str1);  
        System.out.println("Second String: " + str2);  
        System.out.println("Combined String: " + result);  
    }  
}
```

Sample Output:

```
First String: Hello  
Second String: Java  
Combined String: Hello Java
```

Test Case Verification:

Test Case	String 1	String 2	Expected Output	Status
1	Hello	Java	Hello Java	Pass
2	Good	Morning	Good Morning	Pass
3	Computer	Science	Computer Science	Pass

Conclusion: The Java program was successfully executed and the concat() method was demonstrated correctly with all test cases passing.

7. substring() Method

Aim: To extract a portion of a String from a specified starting index.

Definition: The substring() method returns a part of a String from the specified starting index to the end of the String.

Syntax: string.substring(beginIndex);

Program:

```
class SubstringExample {  
    public static void main(String[] args) {  
        String str = "Programming";  
        String result = str.substring(3);  
        System.out.println("Original String: " + str);  
        System.out.println("Substring: " + result);  
    }  
}
```

Sample Output:

```
Original String: Programming  
Substring: gramming
```

Test Case Verification:

Test Case	Input String	Index	Expected	Status
1	Programming	3	gramming	Pass
2	Computer	4	uter	Pass
3	Hello	2	llo	Pass

Conclusion: The Java program was successfully executed and the substring() method was demonstrated correctly with all test cases passing.

8. indexOf() Method

Aim: To find the position (index) of the first occurrence of a character in a String.

Definition: The indexOf() method returns the index of the first occurrence of the specified character or substring. Returns -1 if not found.

Syntax: string.indexOf(character);

Program:

```
class IndexOfExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
        int index = str.indexOf('l');  
        System.out.println("String: " + str);  
        System.out.println("Index of 'l': " + index);  
    }  
}
```

Sample Output:

```
String: Hello  
Index of 'l': 2
```

Test Case Verification:

Test Case	Input String	Character	Expected	Status
1	Hello	l	2	Pass
2	Java	a	1	Pass
3	Computer	p	3	Pass

Conclusion: The Java program was successfully executed and the indexOf() method was demonstrated correctly with all test cases passing.

9. lastIndexOf() Method

Aim: To find the position of the last occurrence of a character in a String.

Definition: The lastIndexOf() method returns the index of the last occurrence of the specified character or substring.

Syntax: string.lastIndexOf(character);

Program:

```
class LastIndexOfExample {  
    public static void main(String[] args) {  
        String str = "Hello";  
        int index = str.lastIndexOf('l');  
        System.out.println("String: " + str);  
        System.out.println("Last index of 'l': " + index);  
    }  
}
```

Sample Output:

```
String: Hello  
Last index of 'l': 3
```

Test Case Verification:

Test Case	Input String	Character	Expected	Status
1	Hello	l	3	Pass
2	Java	a	3	Pass
3	Banana	a	5	Pass

Conclusion: The Java program was successfully executed and the lastIndexOf () method was demonstrated correctly with all test cases passing.

10. replace() Method

Aim: To replace all occurrences of a specified character with another character.

Definition: The replace() method replaces all occurrences of a specified character with another character and returns a new String.

Syntax: string.replace(oldChar, newChar);

Program:

```
class ReplaceExample {
    public static void main(String[] args) {
        String str = "Hello";
        String result = str.replace('l', 'p');
        System.out.println("Original String: " + str);
        System.out.println("After Replacement: " + result);
    }
}
```

Sample Output:

```
Original String: Hello
After Replacement: Heppo
```

Test Case Verification:

Test Case	Input String	Replacement	Expected	Status
1	Hello	l → p	Heppo	Pass
2	Java	a → o	Jovo	Pass
3	Hello	H → Y	Yello	Pass

Conclusion: The Java program was successfully executed and the replace() method was demonstrated correctly with all test cases passing.

11. trim() Method

Aim: To remove leading and trailing whitespace from a String.

Definition: The trim() method removes leading and trailing whitespace (spaces, tabs, etc.) from a String and returns a new String.

Syntax: string.trim();

Program:

```
class TrimExample {  
    public static void main(String[] args) {  
        String str = "  Hello Java  ";  
        String result = str.trim();  
        System.out.println("Original String: [" + str + "]");  
        System.out.println("After trim(): [" + result + "]");  
    }  
}
```

Sample Output:

```
Original String: [ Hello Java ]  
After trim(): [Hello Java]
```

Test Case Verification:

Test Case	Input String	Expected Output	Actual Output	Status
1	" Hello "	Hello	Hello	Pass
2	" Java "	Java	Java	Pass
3	"Computer"	Computer	Computer	Pass

Conclusion: The Java program was successfully executed and the trim() method was demonstrated correctly with all test cases passing.

12. contains() Method

Aim: To check whether a String contains a specified sequence of characters.

Definition: The contains() method checks whether a specified sequence of characters exists in a String. Returns true if found, otherwise false.

Syntax: string.contains(sequence);

Program:

```
class ContainsExample {  
    public static void main(String[] args) {  
        String str = "I love Java";  
        boolean result = str.contains("Java");  
        System.out.println("String: " + str);  
        System.out.println("Contains Java: " + result);  
    }  
}
```

Sample Output:

```
String: I love Java  
Contains Java: true
```

Test Case Verification:

Test Case	Input String	Search Text	Expected	Status
1	I love Java	Java	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Science	true	Pass

Conclusion: The Java program was successfully executed and the contains() method was demonstrated correctly with all test cases passing.

13. startsWith() Method

Aim: To check whether a String starts with a specified sequence of characters (prefix).

Definition: The startsWith() method checks whether a String begins with the specified prefix. Returns true if the String starts with the prefix; otherwise false.

Syntax: string.startsWith(prefix);

Program:

```
class StartsWithExample {  
    public static void main(String[] args) {  
        String str = "Java Programming";  
        boolean result = str.startsWith("Java");  
        System.out.println("String: " + str);  
        System.out.println("Starts with Java: " + result);  
    }  
}
```

Sample Output:

```
String: Java Programming  
Starts with Java: true
```

Test Case Verification:

Test Case	Input String	Prefix	Expected	Status
1	Java Programming	Java	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Computer	true	Pass

Conclusion: The Java program was successfully executed and the startsWith() method was demonstrated correctly with all test cases passing.

14. endsWith() Method

Aim: To check whether a String ends with a specified sequence of characters (suffix).

Definition: The endsWith() method checks whether a String ends with the specified suffix. Returns true if the String ends with the suffix; otherwise false.

Syntax: string.endsWith(suffix);

Program:

```
class EndsWithExample {  
    public static void main(String[] args) {  
        String str = "Java Programming";  
        boolean result = str.endsWith("Programming");  
        System.out.println("String: " + str);  
        System.out.println("Ends with Programming: " + result);  
    }  
}
```

Sample Output:

```
String: Java Programming  
Ends with Programming: true
```

Test Case Verification:

Test Case	Input String	Suffix	Expected	Status
1	Java Programming	Programming	true	Pass
2	Hello World	Java	false	Pass
3	Computer Science	Science	true	Pass

Conclusion: The Java program was successfully executed and the endsWith() method was demonstrated correctly with all test cases passing.

7. Compilation and Execution

All programs were compiled and executed from the command line (or IDE terminal) as follows:

Compilation:

```
javac CharAtExample.java
javac UpperCaseExample.java
... (for each class)
```

Execution:

```
java CharAtExample
java UpperCaseExample
... (for each class)
```

No compilation errors or runtime exceptions were encountered. All programs produced the expected console output.

8. Sample Output (Summary)

Representative sample outputs for each method are shown in the individual method sections above. All outputs matched the expected results documented in the test-case tables.

9. Test Cases / Verification

A total of **42 test cases** (3 per method × 14 methods) were executed. Every test case passed successfully, confirming correct implementation and understanding of each String method. Detailed test-case tables appear under each method.

10. Result / Conclusion

All fourteen core String methods of the Java language were successfully studied, implemented and verified. The experiment demonstrated:

- Character retrieval by index (charAt)
- Case conversion (toUpperCase / toLowerCase)
- Content-based and case-insensitive comparison (equals / equalsIgnoreCase)
- String joining (concat)
- Sub-string extraction (substring)
- Forward and reverse searching (indexOf / lastIndexOf)
- Character replacement and whitespace removal (replace / trim)
- Presence and boundary checking (contains / startsWith / endsWith)

The laboratory exercise confirms that the String class provides a rich, immutable API for common text-processing tasks. All programs compiled cleanly, produced correct output, and the associated test cases passed with 100 % success rate. This completes the laboratory manual on Java String Methods.

End of Laboratory Report — 14 String Methods demonstrated with working programs, sample outputs and verified test cases.